

Real-Time Intrusion Detection in Internet of Vehicles for Consumer Autonomous Intelligent Systems Using Hyper-Dimensional Computing

Xiaoming He[✉], *Member, IEEE*, Da Tang, Haodong Lu, Yunzhe Jiang[✉], Hadeel Alsolai[✉],
and Shahid Mumtaz[✉], *Senior Member, IEEE*

Abstract—With the rapid development of consumer autonomous intelligent systems, ensuring the security of Internet of Vehicles (IoV) has become a critical challenge. Machine Learning (ML) and Deep Learning (DL) techniques are widely used to develop classifier-based Intrusion Detection Systems (IDS). However, traditional methods often suffer from high computational complexity, limiting their effectiveness in real-time detection. Hyperdimensional Computing (HDC), a brain-inspired machine learning paradigm, offers a compelling combination of high precision with exceptional robustness and training efficiency. In this paper, we present Hyperdimensional Intrusion Detection in IoV (HIDIV), a lightweight and efficient framework designed explicitly for IoV security. HIDIV introduces a dynamic error update module, enabling faster convergence and higher accuracy than conventional HDC methods. Experimental results show that HIDIV significantly accelerates training and inference, achieving speedups of approximately 43.4% and 33.2% over state-of-the-art machine learning methods while maintaining comparable accuracy. Furthermore, the proposed framework also surpasses traditional HDC in terms of accuracy.

Index Terms—Internet of Vehicles, consumer autonomous intelligent systems, intrusion detection, hyper-dimensional computing (HDC).

I. INTRODUCTION

THE Internet of Vehicles (IoV) has transformed modern vehicles into interconnected intelligent nodes capable of

real-time communication with other vehicles, infrastructure, and consumer autonomous intelligent systems [2]. While this connectivity enables advanced functionalities such as autonomous driving and real-time traffic management, it also introduces significant security challenges [3]. The expanded attack surface of IoV systems makes them vulnerable to cyber threats like Denial of Service (DoS), spoofing, and malware injection, which can compromise vehicle safety and disrupt transportation networks. Traditional security mechanisms, *e.g.*, firewalls and encryption, are often inadequate in these dynamic and resource-constrained environments. As a result, Artificial Intelligence (AI)-based Intrusion Detection Systems (IDS) have emerged as critical solutions for enhancing the security of IoV networks [4].

AI-based IDS techniques can be categorized into traditional Machine Learning (ML) and Deep Learning (DL) approaches. Conventional ML algorithms, *e.g.*, Support Vector Machines (SVM) [5], Random Forests [6], and Naive Bayes [7], have been extensively used for network traffic classification and anomaly detection. While these approaches provide reliable performance, they often rely on extensive feature engineering and face challenges in adapting to the dynamic and evolving nature of cyber threats in IoV environments. In contrast, DL approaches, *e.g.*, Convolutional Neural Networks (CNNs) [8] and Recurrent Neural Networks (RNNs) [9], have demonstrated superior performance by automatically extracting features from raw network data, making them well-suited for intrusion detection. For instance, Yang and Shami [10] developed a CNN-based IDS that achieved high detection rates for various attacks in vehicular networks. Jablaoui et al. [9] illustrated the effectiveness of RNNs in detecting sequential attack patterns. However, DL-based approaches in IoV encounter several challenges, such as high computational demands, imbalanced datasets, and difficulty in achieving real-time performance. These challenges highlight the need for more efficient, adaptive, and resilient solutions designed to meet the specific constraints of IoV systems.

Hyper-dimensional Computing (HDC), a brain-inspired machine learning paradigm, has demonstrated significant potential for intrusion detection in the IoV domain [11], [12], [13]. Its key advantages include: (1) **Efficiency and Real-Time Performance**: IoV applications, particularly in autonomous and cooperative driving, require ultra-low latency, where even millisecond delays can have critical consequences. HDC

Received 1 September 2025; revised 1 November 2025; accepted 7 December 2025. Date of publication 15 December 2025; date of current version 25 March 2026. This work was supported in part by the National Science Research Start-Up Foundation of Recruiting Talents of Nanjing University of Posts and Telecommunications (NUPT) under Grant NY223188. An earlier version of this paper was presented in part at the 2025 IEEE/CIC International Conference on Communications in China (ICCC), Shanghai, China, August 2025 [DOI: 10.1109/ICCC65529.2025.11148923]. (*Corresponding author: Yunzhe Jiang.*)

Xiaoming He and Da Tang are with the College of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: hexiaoming@njupt.edu.cn; 1023071818@njupt.edu.cn).

Haodong Lu is with the School of Microelectronics, Fudan University, Shanghai 200433, China (e-mail: haodonglu@fudan.edu.cn).

Yunzhe Jiang is with the College of Communication and Information Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: jiangyunzhe0822@gmail.com).

Hadeel Alsolai is with the Department of Information Systems, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, P.O. Box 84428, Riyadh 11671, Saudi Arabia (e-mail: haalsolai@pnu.edu.sa).

Shahid Mumtaz is with the Department of Engineering, Nottingham Trent University, NG1 4FQ Nottingham, U.K., and also with the Department of Electronic Engineering, Kyung Hee University, Seoul 130-701, South Korea (e-mail: dr.shahid.mumtaz@ieee.org).

Digital Object Identifier 10.1109/TCE.2025.3644450

offers low computational complexity with fast training and inference speeds, enabling real-time network traffic analysis on in-vehicle or edge devices for timely anomaly detection and mitigation. (2) **Adaptability to Resource-Constrained Environments:** IoV components, e.g., in-vehicle units and roadside infrastructure, often operate with limited computational, storage, and energy resources. While traditional DL models are resource-intensive, HDC models are compact and lightweight, making them suitable for deployment in constrained environments. (3) **Robustness and Noise Tolerance:** IoV network data is often noisy or incomplete due to wireless interference, packet loss, or hardware failures. Operating in high-dimensional representations, HDC inherently exhibits strong robustness, maintaining high detection accuracy even in the presence of data imperfections.

Despite these advantages, existing HDC approaches exhibit notable limitations. First, their encoding schemes are typically static during training, with updates confined to class hypervectors. This necessitates the use of extremely high-dimensional representations to maintain accuracy [14]. Although methods such as AdaptHD [15] introduce adaptive learning-rate adjustment strategies to accelerate convergence, they primarily focus on optimizing the training schedule while leaving the encoding process unchanged. Second, the training process often converges slowly, demanding a large number of iterations and high-dimensional vectors, which is impractical in resource-constrained and time-sensitive IoV environments [16]. These limitations underscore the need for a more adaptive encoding mechanism that can simultaneously achieve high detection accuracy and real-time performance in IoV intrusion detection systems.

To address these challenges, this paper proposes Hyperdimensional Intrusion Detection in IoV (HIDIV), a novel HDC training architecture that dynamically updates the encoder to achieve higher training efficiency. HIDIV enables attack detection in real-time and resource-limited IoV environments, overcoming the scalability limitations of conventional methods. The main contributions are as follows:

- **Integration of HDC for Intrusion Detection in IoV:** We present the first application of hyperdimensional computing (HDC) to cybersecurity in vehicular networks. By leveraging HDC's inherent robustness and computational efficiency, our framework enables real-time anomaly detection in consumer-grade autonomous systems.
- **Dynamic Encoder Update Mechanism:** We propose a novel online encoder update algorithm that dynamically refines hypervector representations during training. This adaptive approach overcomes the limitations of static encoding, achieving faster convergence and improved classification accuracy.
- **Inference Optimization and Hardware Acceleration:** To meet the latency and energy constraints of IoV systems, we optimize the HDC inference pipeline through algorithmic enhancements and implement it on an FPGA. Our custom hardware architecture enables efficient,

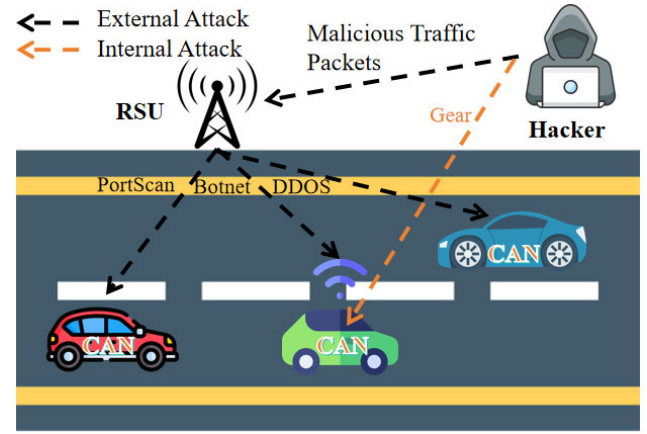


Fig. 1. Attack scenario in IoV.

low-power, and real-time intrusion detection, making it suitable for edge deployment.

- **Extensive Empirical Validation:** Extensive experiments on benchmark vehicular intrusion datasets demonstrate that HIDIV achieves competitive detection accuracy while requiring significantly less training time. Furthermore, HIDIV outperforms conventional deep learning models in both inference speed and training efficiency, validating its practical effectiveness.

The paper is organized as follows: Section II covers Related Work; Section III details the Preliminary; Section IV designs the HDC model; Section V describes the Inference Acceleration; Section VI presents evaluation results and analysis; and Section VII concludes the paper.

II. RELATED WORK

A. Intrusion Detection

In IoV, vehicular network systems are required to address both internal and external network threats simultaneously. Internal attacks primarily target the Controller Area Network (CAN) bus, where attackers can directly inject malicious messages into connected intelligent vehicles to disrupt their normal operations. Common internal attacks include Fuzzing attacks, Gear spoofing, and Revolutions Per Minute (RPM) spoofing, which may compromise vehicle safety by manipulating critical systems such as braking or engine control. Additionally, hackers can launch external attacks by controlling Road Side Units (RSUs) to send malicious traffic to vehicles, such as Distributed Denial of Service (DDoS), Port Scanning, Brute Force, Botnet, and Web Attacks, which share similarities with traditional IoT environments. Therefore, as illustrated in Fig. 1, the proposed HIDIV must be capable of effectively mitigating various types of both internal and external attacks.

B. Traditional AI Model for Intrusion Detection

Deep learning models have been widely applied to in-vehicle network intrusion detection due to their capability to extract complex temporal and spatial features. Wang et al. [17] provided a comprehensive survey of deep learning-based methods, discussing models such as CNNs,

RNNs, and autoencoders for CAN message classification. To improve representation learning, Gao et al. [18] proposed a tokenization-based deep learning framework that enhances the sequential modeling of CAN traffic using textual embeddings. In parallel, optimization-driven detection frameworks have emerged to address trade-offs between detection accuracy and system constraints. Zhang et al. [19] formulated intrusion detection as a many-objective optimization problem, balancing performance across multiple security metrics. Similarly, Wang et al. [20] developed a multi-sensor IDS that fuses spatial and temporal signals to increase robustness in autonomous driving scenarios. While these approaches demonstrate high accuracy, their deep architectures and iterative optimization mechanisms lead to significant inference latency and memory usage, limiting deployment on real-time embedded platforms.

C. Hybrid Computing for Intrusion Detection

To overcome data locality and privacy concerns, distributed detection paradigms have been proposed. Qin et al. [21] introduced CVMIDS, a cloud-vehicle collaborative system where cloud-based models support edge-level detection. Qu and Wang [22] presented FEDSA-ResNetV2, which uses federated learning to collaboratively train models across vehicles without sharing raw data. The integration of blockchain with federated learning further strengthens trust and security. Abou El Houda et al. [23] designed a blockchain-enabled federated IDS that ensures secure model aggregation and tamper-resistance in vehicular edge computing. Similarly, Xing et al. [24] applied blockchain to coordinate collaborative intrusion detection among connected vehicles. Despite improvements in detection, these distributed systems introduce communication overhead, synchronization issues, and require infrastructure support, which can undermine real-time responsiveness in high-mobility environments.

Hybrid intrusion detection systems aim to improve resilience by combining multiple detection paradigms. Yang et al. [25] proposed MTH-IDS, a multitiered hybrid architecture that integrates anomaly-based and rule-based components across multiple processing layers. In a similar vein, Elsayed et al. [26] presented AdaptIDS, which adaptively reconfigures its detection behavior based on mission-critical system status, providing flexibility under dynamic conditions. Additionally, semantic reasoning has been introduced via structured knowledge models. Sun et al. [27] proposed KG-ID, a knowledge graph-based IDS that leverages entity relationships and context information to identify subtle or stealthy intrusions beyond statistical signatures. While these models show enhanced interpretability and robustness, they are often associated with high system complexity, intensive reasoning steps, and limited real-time applicability on vehicular-grade hardware.

D. Our Motivation

The studies mentioned above have laid a strong foundation for intrusion detection in the Internet of Vehicles. However, they often face challenges in real-time deployment,

particularly in latency-sensitive and resource-constrained environments. To address this, we propose a novel intrusion detection system based on HDC. Our method encodes input features into high-dimensional binary vectors and employs ultra-efficient similarity search, allowing for: (1) Real-time classification with deterministic latency, (2) Low storage and computational overhead, (3) Suitability for deployment on in-vehicle embedded systems.

III. PRELIMINARY

A. Encoding

The encoding mechanism in HDC is inspired by how the human brain processes information, mapping data into a high-dimensional space. Specifically, samples are combined with randomly generated high-dimensional base vectors to create a high-dimensional vector, known as a *hypervector*. A *hypervector* typically contains thousands of elements, each uniformly carrying the entire information of the sample, rather than having specific elements responsible for storing particular pieces of information. This holographic distribution ensures high redundancy and robustness, allowing the *hypervector* to effectively handle noise and partial information loss.

The selection of an encoding method is influenced by the data type. Nevertheless, the core objective remains consistent, *i.e.*, to preserve the distance correlations present in the original data, thereby ensuring that the encoded hypervectors accurately capture its structure and underlying relationships. Common encoding methods include multiplication with random projections, fractional binding, and specialized techniques for time-series data. These methods are based on the fundamental operations of HDC, aiming to efficiently capture and retain the key characteristics of the data [28].

B. Basic Operations in HDC

HDC is built upon the concept of high-dimensional vectors, typically consisting of thousands of dimensions. These vectors, represented as $\mathbf{v} \in \mathbb{R}^D$, where D denotes dimensionality (e.g., $D = 10,000$), are often sparse and generated randomly. The high dimensionality ensures vectors remain nearly orthogonal to each other, resulting in significant vector differences from even minor changes in input data. This characteristic grants HDC robustness against noise and the capability to effectively handle incomplete or corrupted data, which is particularly beneficial in real-world scenarios with varying data quality.

The fundamental operations of HDC include: (1) **Binding**: Binding combines two vectors, $\mathbf{v}_1$ and $\mathbf{v}_2$, into a new vector $\mathbf{v}_{\text{bind}}$. This operation typically employs element-wise multiplication, such as XOR for binary vectors or component-wise multiplication for real-valued vectors: $\mathbf{v}_{\text{bind}} = \mathbf{v}_1 \otimes \mathbf{v}_2$, where $\otimes$ represents the binding operator. Binding encodes relationships between entities, such as associating a specific feature with its corresponding label. (2) **Bundling**: Bundling aggregates multiple vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$ into a single vector $\mathbf{v}_{\text{bundle}}$. It typically utilizes element-wise addition: $\mathbf{v}_{\text{bundle}} = \sum_{i=1}^n \mathbf{v}_i$. Bundling is particularly effective for representing collections or sets of items, enabling the grouping of similar data points. (3) **Similarity Check**: Similarity check assesses the

degree of similarity between two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$. Common similarity metrics include cosine similarity for real-valued vectors: $\text{sim}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|}$, and Hamming distance for binary vectors: $\text{sim}(\mathbf{v}_1, \mathbf{v}_2) = 1 - \frac{\text{Hamming}(\mathbf{v}_1, \mathbf{v}_2)}{D}$, where $\text{Hamming}(\mathbf{v}_1, \mathbf{v}_2)$ counts the number of differing bits between the two vectors. Similarity checking is essential in applications such as classification and information retrieval, where identifying patterns or matching data points based on their similarities is fundamental.

C. Challenges

Despite its simplicity and scalability, HDC-based models face two key challenges in deployment: (1) Static Encoding Limitation: The lack of adaptation in encoding limits classification accuracy in evolving data distributions. (2) Hardware Mapping Efficiency: The computational benefits of HDC are not fully exploited without an efficient hardware implementation, especially for latency-critical applications. Field Programmable Gate Arrays (FPGAs) offer a compelling solution due to their inherent parallelism, reconfigurability, and low power consumption. In this work, we leverage these properties to design a custom FPGA architecture that accelerates the inference stage of our proposed HIDIV system. The proposed design exploits the structured nature of HDC computation to achieve both low-latency and low-power performance, suitable for deployment in vehicular edge devices.

IV. HDC MODEL DESIGN

A. Overview of HIDIV

The HIDIV framework, shown in Fig. 2, presents an effective approach to address the unique security challenges in IoV environments. By leveraging HDC, HIDIV transforms raw IoV data into high-dimensional hypervectors, enabling robust, efficient, and scalable intrusion detection. The architecture consists of three key components: (1) *Encoding Module*: Maps input data into high-dimensional hypervectors, facilitating efficient representation and processing. (2) *Training module*: Comprises two stages, *i.e.*, initial training and retraining. These steps construct and iteratively refine class hypervectors, which serve as the trained model for intrusion detection. (3) *Inference module*: Performs real-time classification through optimized similarity computations, ensuring fast and accurate attack detection. The HIDIV training process is shown in Alg. 1.

B. HDC Encoding for IoV Data

HDC is known for its robustness, noise tolerance, and suitability for learning from high-dimensional data representations. These characteristics make it particularly advantageous in dynamic and noisy IoV environments. The encoding module in HIDIV maps low-dimensional input features into high-dimensional space, generating *hypervectors* (HVs) that capture the critical characteristics of the data. Given an input sample $\mathbf{x} = \{v_1, v_2, \dots, v_F\}$, where F is the number of features, the encoding process projects $\mathbf{x}$ into a D -dimensional space (with $D \gg F$) using a projection matrix $\mathbb{P} = \{P_1, P_2, \dots, P_F\}^D$ as illustrated in Fig. 2 (●). The projection matrix is sampled

from a Gaussian distribution, ensuring that the encoded hypervectors are high-dimensional and stochastic, which enhances robustness to noise. The encoded hypervector $\mathcal{H}$ is then computed as:

$$\mathcal{H} = \text{relu}(v_1 \times P_1 + v_2 \times P_2 + \dots + v_F \times P_F), \quad (1)$$

where $\text{relu}(\cdot)$ denotes the Rectified Linear Unit (ReLU) activation function. This non-linear transformation introduces sparsity and enhances the representational capacity of the hypervector, ensuring a compact yet expressive encoding of the input data.

Algorithm 1 Training Procedure of HIDIV

Require: Data x in a training dataset

- 1: **● Encoding**
 - 2: $\mathcal{H} \leftarrow \text{relu}(\mathbb{P} \otimes \mathbf{x})$
 - 3: **● Similarity calculation**
 - 4: $s \leftarrow \text{softmax}(\delta(\mathbb{C}, \mathcal{H}))$
 - 5: **● Error separation**
 - 6: $e \leftarrow y - s$
 - 7: **● Update class hypervectors** $\mathbb{C}$
 - 8: $\mathbb{C} \leftarrow \mathbb{C} \oplus \lambda \mathcal{H} e$
 - 9: **● Update projection matrix** $\mathbb{P}$
 - 10: $\text{update}^+ \leftarrow \{e_i \mid e_i > 0\} \cdot \lambda \cdot y \odot \mathbb{B}$
 - 11: $\text{update}^- \leftarrow \text{mean}\{e_i \mid e_i < 0\} \cdot \lambda \cdot y \odot \mathbb{B}$
 - 12: $\text{update} \leftarrow \text{update}^+ + \text{update}^-$
 - 13: $\mathbb{P} \leftarrow \mathbb{P} \oplus \text{update}$
-

C. Initial Training for Intrusion Detection

During the initial training, class hypervectors are constructed by aggregating the hypervectors corresponding to training samples of the same class. Specifically, given m training samples, each represented by a hypervector $\mathcal{H}_i$ with a class label C_j , the class hypervector C_j is computed as:

$$C_j = \sum_{i=1}^m \mathcal{H}_i \quad (\text{where } \mathcal{H}_i \text{ belongs to class } C_j). \quad (2)$$

During training, the class hypervectors are iteratively refined to correct misclassifications:

$$C_j = C_j + \lambda \cdot [1 - \delta(\mathcal{H}, C_j)] \times \mathcal{H} \quad (3)$$

$$C_{j'} = C_{j'} - \lambda \cdot [1 - \delta(\mathcal{H}, C_{j'})] \times \mathcal{H} \quad (4)$$

where λ is the learning rate, controlling the magnitude of updates, and $\mathcal{H}$ is the hypervector of a misclassified sample. $C_{j'}$ represents the class hypervector corresponding to the incorrect prediction. When a sample with true label j is misclassified as j' , the class hypervector C_j is reinforced to improve correct classification, while $C_{j'}$ is adjusted to reduce the misclassification.

D. Retraining With Dynamic Encoding

The initial training described above does not update the projection matrix, which may hinder HDC's training efficiency. Without updates, the projection matrix might fail to capture essential data features, potentially resulting in slower

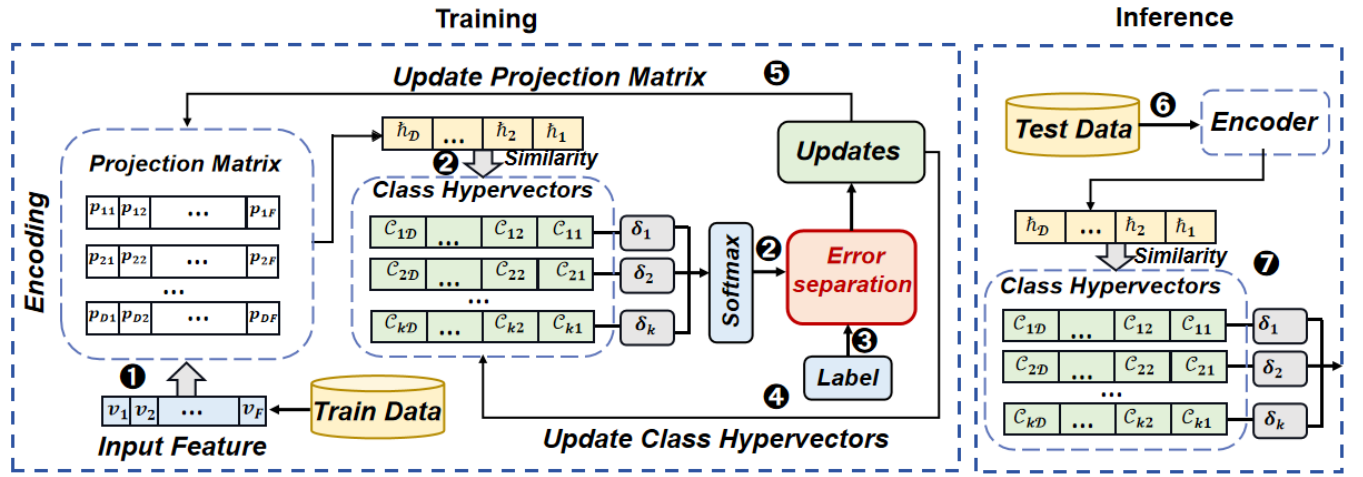


Fig. 2. Overview of HIDIV.

convergence or degraded performance. To overcome the limitations of static encoding, we introduce a dynamic retraining approach. This strategy incorporates adaptive updates to the projection matrix $\mathbb{P}$ in addition to updating class hypervectors, allowing the encoder to generate hypervectors that better align with classification objectives.

1) *Update Class Hypervectors*: During retraining, the encoded hypervector $\mathcal{H}$ is compared against all class hypervectors $\mathcal{C}$ and normalized using the *softmax* function (2). The per-class error e is then computed based on the ground truth label y and the normalized similarity scores s (3). Finally, each class hypervector is updated by incorporating $\mathcal{H}$, scaled by e and the learning rate λ (4).

2) *Update Projection Matrix*: A companion matrix $\mathbb{B}$ is initialized using the He uniform initializer [29]: $\mathbb{B} \sim \mathcal{U}\left(-\sqrt{\frac{6}{F}}, +\sqrt{\frac{6}{F}}\right)$, and remains fixed during retraining. The projection matrix $\mathbb{P}$ is updated by binding it with $\mathbb{B}$, scaled according to the per-dimension error and learning rate λ . To further accelerate convergence, the model explicitly separates positive feedback (from correctly classified dimensions) from negative feedback (from erroneous classifications), thereby facilitating faster and more stable optimization (5).

During the retraining phase, the same dataset as in the initial training is employed. Instead of repeating training, the model adaptively updates the projection matrix to refine feature representations. This dynamic adjustment mitigates overfitting by preventing memorization of training samples, while maintaining consistency between the projection matrix and the input feature space, thereby improving the encoder's adaptability to the classification objective.

E. HDC Inference for Intrusion Detection

As shown in Fig. 2 (6 and 7), the inference phase in the HIDIV framework utilizes the trained class hypervectors and the dynamically updated projection matrix to classify incoming IoV data as either normal behavior or an intrusion. This phase relies on a similarity-based classification mechanism, in which an encoded input hypervector $\mathcal{H}$ is compared against a set of class hypervectors $\mathcal{C} = \{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_k\}$, where k

denotes the total number of classes (6). The predicted class label for the input data corresponds to the class with the highest similarity score (7). The similarity between $\mathcal{H}$ and each class hypervector $\mathcal{C}_j$ is computed using cosine similarity:

$$\text{Similarity}(\mathcal{H}, \mathcal{C}_j) = \frac{\mathcal{H} \cdot \mathcal{C}_j}{\|\mathcal{H}\| \|\mathcal{C}_j\|}. \quad (5)$$

To enhance the efficiency of the inference process, particularly in real-time IoV environments where computational resources may be limited, two key optimizations are introduced: (1) *Precomputation of Class Hypervector Norms*: The norm $\|\mathcal{C}_j\|$ of each class hypervector is computed during the training phase and remains fixed throughout the inference. As these norms do not change, we precompute and store them to avoid redundant calculations during runtime, thus reducing computational overhead. (2) *Elimination of Input Hypervector Norm*: Since classification decisions are based solely on the relative similarity scores among the class hypervectors, the norm $\|\mathcal{H}\|$ of the input hypervector can be omitted without affecting the outcome. This simplification further reduces the required operations, especially in high-throughput settings. Applying these optimizations, the cosine similarity computation reduces to a dot product:

$$\text{Similarity}(\mathcal{H}, \mathcal{C}_j) = \mathcal{H} \cdot \mathcal{C}_j. \quad (6)$$

This simplified inference process enables HIDIV to perform rapid and accurate classification, making it well-suited for deployment in real-time, resource-constrained IoV systems.

V. INFERENCE ACCELERATION

A. Overview

This section presents the inference acceleration strategy for the trained HDC-based network intrusion detection model, focusing on optimizing associative search and its deployment on FPGA hardware. In a typical HDC classifier, inference involves a similarity search between the encoded query hypervector and each class hypervector, which can be computationally expensive in resource-constrained edge scenarios such as consumer electronics. To address this challenge,

we propose an optimization method that clusters the hyper-vectors associated with each class and replaces the original class prototypes with representative cluster centroids. This significantly reduces the number of similarity computations and enables efficient memory indexing. The compressed structure is inherently well-suited for parallelism and deterministic execution, making it highly compatible with FPGA architectures. By conducting HDC encoding and associative search on FPGA, we achieve substantial speedups and energy efficiency without compromising detection accuracy.

B. Associative Search Optimization

In the context of real-time intrusion detection on consumer electronics, efficient inference is paramount due to limited computational and power resources. Our proposed system, HIDIV, accelerates inference by optimizing the associative search step of the hyperdimensional classifier through a value-clustering mechanism that minimizes redundant computation while preserving detection accuracy.

During standard inference in HDC, the class prediction is made by computing similarity scores, typically cosine similarity, between an input query hypervector $\mathbf{Q} = \{q_1, q_2, \dots, q_D\}$ and each class prototype $\mathbf{C}^i = \{w_1^i, w_2^i, \dots, w_D^i\}$, where D is the hypervector dimensionality. Cosine similarity can be reduced to a dot product operation when all vectors are normalized, which is feasible in our case, as class vectors are precomputed offline. Thus, inference reduces to:

$$\text{sim}(\mathbf{Q}, \mathbf{C}^i) = \sum_{j=1}^D q_j \cdot w_j^i. \quad (7)$$

Although each dot product involves D multiplications and additions, the structured nature of HIDIV's class hypervectors enables us to significantly reduce the computational burden.

1) *Cluster-Based Class Vector Compression*: We introduce a cluster-aware representation for each class prototype to exploit the redundancy in value distributions across dimensions. Specifically, HIDIV restricts the set of possible values w_j^i in each class vector to a compact set of representative centroids $\{c_1^i, c_2^i, \dots, c_k^i\}$, where $k \ll D$. These centroids are obtained by applying the k-means clustering algorithm to the set of dimension-wise class values:

$$\{w_1^i, w_2^i, \dots, w_D^i\} \in \{c_1^i, c_2^i, \dots, c_k^i\}. \quad (8)$$

Formally, the clustering minimizes the intra-cluster variance over the original set of class vector values θ^i , for class i :

$$\min_{\{c_j\}_{j=1}^k} \sum_{j=1}^k \sum_{w \in c_j} \|w - c_j\|^2. \quad (9)$$

This approximation enables us to transform the dot product into a two-phase computation: (1) accumulate the query values that correspond to identical cluster centroids in the class vector; (2) multiply each accumulated sum by its associated centroid value and sum the products:

$$\text{sim}(\mathbf{Q}, \mathbf{C}^i) = \sum_{m=1}^k s_m^i \cdot c_m^i \quad \text{where} \quad s_m^i = \sum_{j \in \mathcal{I}_m^i} q_j, \quad (10)$$

where $\mathcal{I}_m^i$ is the set of indices in class i whose values are assigned to centroid c_m^i . As a result, the number of multiplications per similarity computation drops from D to k .

2) *Robustness-Aware Model Compression*: To ensure that the associative search optimization in HIDIV maintains reliable detection performance for network intrusion tasks in consumer electronics, we introduce a robustness-aware compression framework. This method balances computational efficiency with classification accuracy by iteratively evaluating and refining the clustered class hypervectors.

After training, HIDIV replaces the raw class hypervector elements with a reduced set of representative centroids obtained through dimension-wise clustering. While this step significantly reduces the number of similarity computations, it introduces approximation error that may affect classification performance. To quantify this impact, we evaluate the validation accuracy of the clustered model and define the degradation metric ΔE as:

$$\Delta E = E_{\text{clustered}} - E_{\text{original}} \quad (11)$$

where E_{original} and $E_{\text{clustered}}$ are the validation error rates before and after clustering, respectively. This evaluation is conducted on a hold-out validation subset of the training data. In practice, we observe that small values of k (number of clusters) can maintain accuracy within an acceptable tolerance, while offering significant computational savings.

If the accuracy drop ΔE exceeds a user-defined threshold ϵ , we activate a lightweight retraining procedure designed to align the class hypervectors with their clustered counterparts. Specifically, for each training sample with query vector $\mathbf{Q}$, HIDIV evaluates its similarity to the current class hypervectors: (1) If $\mathbf{Q}$ is correctly classified, no action is taken. (2) If misclassification occurs, and $\mathbf{Q}$ is wrongly assigned to class i while its true label is class j , the class vectors are updated as follows:

$$\mathbf{C}^i \leftarrow \mathbf{C}^i - \mathbf{Q}, \quad \mathbf{C}^j \leftarrow \mathbf{C}^j + \mathbf{Q}. \quad (12)$$

This update reinforces correct associations while penalizing false ones, allowing the class prototypes to adapt to the clustered structure. After each retraining iteration, clustering is reapplied to the updated class vectors, and the validation error ΔE is re-evaluated.

This retraining-compression loop continues until the quality constraint $\Delta E < \epsilon$ is met or a maximum iteration count is reached. In our experiments, early integration of clustering into the training loop yields convergence in a few iterations, allowing the final model to retain accuracy while being optimized for index-based search and FPGA deployment.

This adaptive strategy ensures that HIDIV achieves a compact and hardware-friendly model representation without compromising its ability to accurately detect subtle attack patterns in real-world traffic.

C. Encoding Acceleration

The first stage of HDC involves encoding the input feature vector into a high-dimensional query hypervector. In this work, we adopt the classical projection matrix approach

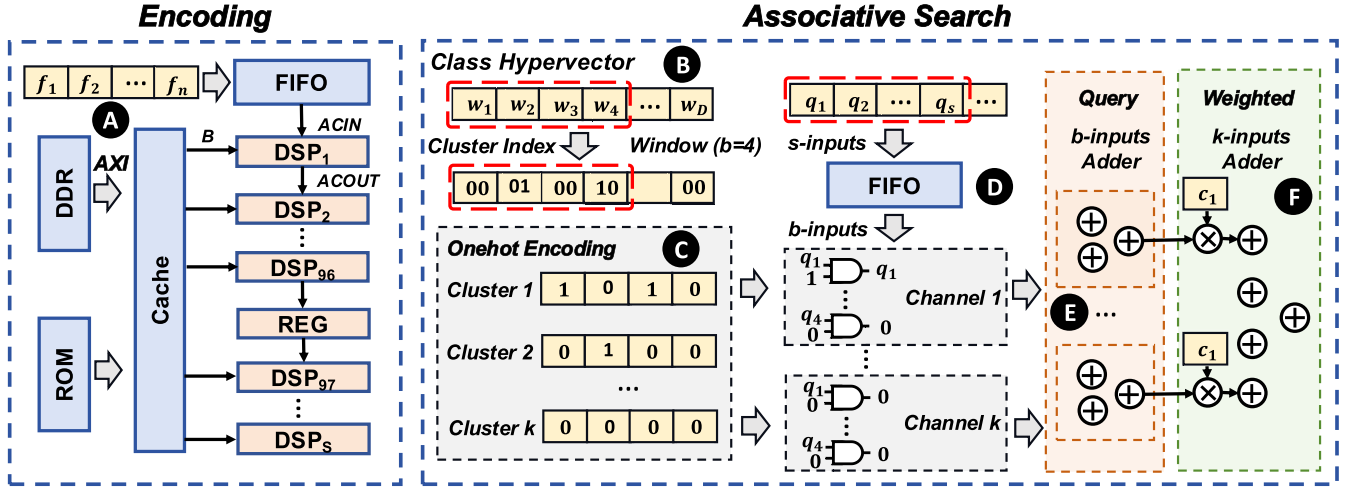


Fig. 3. Hardware architecture design for efficient encoding and associative search operations.

for encoding, rather than relying on basis vector substitution. The encoding module is implemented using a cascaded Digital Signal Processing (DSP) structure, which requires dedicated internal cascade ports to support stable high-speed data transmission. However, these proprietary high-throughput interconnects typically impose constraints on the maximum number of DSP blocks that can be cascaded. For example, only up to 96 DSPs can be linked in Xilinx 5E series FPGAs. To fully exploit all available high-speed channels, our design inserts intermediate registers between each 96-DSP cascade block, enabling seamless connectivity to subsequent groups while preserving timing integrity.

Since both the encoding and associative memory modules adopt a pipelined architecture, the latency for processing a single input feature vector is dominated by the encoding stage. Given a feature vector of dimension F , a query hypervector of dimension D , and S DSP blocks instantiated in the encoder, the total number of computation cycles T can be approximated as $T \approx \frac{F \times D}{S}$.

While increasing S improves computational parallelism and accelerates encoding, it also results in higher DSP resource consumption. In addition to computing resources, memory requirements for storing the projection matrix present another major challenge. Assuming a quantization precision of Q bits, the total storage required for the projection matrix reaches $Q \times F \times D$ bits. On-chip Block Random Access Memory (BRAM) alone may not suffice to accommodate this memory footprint. To mitigate potential on-chip memory limitations, we adopt a hybrid memory architecture that leverages both off-chip DDR4 memory and on-chip BRAM to store the projection matrix. Efficient partitioning between on-chip and off-chip resources, along with careful tuning of the AXI bus bandwidth, is critical to ensure that memory access latency does not become a bottleneck for encoding throughput.

D. Associative Search Acceleration

After applying the k -means clustering algorithm to each class hypervector, the vector is partitioned into k cluster centroids. Each element within a class is assigned a cluster index based on its nearest centroid. For example, if the first

element of class 1 is assigned to the first cluster, its index is represented as “0”. In general, dividing a class into k clusters requires each index to be encoded using $\log_2 k$ bits. Therefore, the total storage requirement for the index of a single class is $D \times \log_2 k$ bits, where D is the dimension of the hypervector.

However, the stored cluster index for each element does not encode its class membership; instead, it reflects the element’s original position within the class hypervector. Consequently, to preserve positional mapping, each class requires $D \times \log_2 D$ bits of storage for the indices. Since $D \gg k$, this design significantly reduces the storage overhead compared to directly storing full-precision vector values.

At the end of each encoding cycle, S query elements must undergo similarity computation. As discussed previously, a larger S increases computational throughput. However, this also places stricter timing constraints on the associative search module, which must complete its operations within approximately n clock cycles to maintain pipeline synchronization with the encoding stage.

To prevent idle cycles and ensure efficient resource utilization, the number of query elements processed per cycle must be limited. Let b denote the number of query elements handled per clustering operation. Thus, the total number of cycles required to process all S elements is $\frac{S}{b}$, which must satisfy the condition $\frac{S}{b} \leq F$. This constraint guarantees that the associative search module operates efficiently without becoming a performance bottleneck.

Within each cycle, the b selected cluster indices are distributed across k channels, each corresponding to one of the k clusters. Each channel includes a one-hot encoder that converts the b binary indices (each of width $\log_2 k$ bits) into b -bit one-hot vectors (step C). These one-hot vectors serve as gating masks for bitwise AND operations (step D), selectively passing the corresponding b query elements to their respective channels. Following the gating stage, each channel aggregates its b active query elements using an adder tree (step E). The summed values from each channel are then scaled by their associated cluster weight coefficients $\{c_1, c_2, \dots, c_k\}$ using parallel multipliers. The final similarity score (step F) for the

TABLE I

DATASETS USED FOR EVALUATING HIDIV. (n : NUMBER OF FEATURES, k : NUMBER OF CLASSES)

	n	k	Train Size	Test Size	Release Year
Car-Hacking	9	5	114,585	57,292	2018 [30]
Survival Analysis	9	4	791,552	338,849	2018 [31]
CICIDS2017	77	15	1,321,301	566,272	2017 [32]

class is obtained by summing the weighted outputs across all channels.

VI. EVALUATION AND ANALYSIS

A. Experimental Setup

We implement the proposed HIDIV framework and baseline models on an Intel Xeon Platinum 8255C CPU and an NVIDIA Tesla T4 GPU. All experiments are conducted using the PyTorch framework, with GPU acceleration employed to enhance computational efficiency.

Performance is evaluated on three widely used benchmark datasets for vehicular network security, *i.e.*, the Car-Hacking dataset, the Survival Analysis Dataset for automobile IDS, and the CICIDS2017 dataset. The Car-Hacking and Survival Analysis datasets are designed to simulate internal attack scenarios targeting the CAN bus, with the latter providing driving data collected from three distinct vehicle types. In contrast, the CICIDS2017 dataset is constructed from real-world network traffic and is well-suited for simulating external attack scenarios involving IoT device nodes. By leveraging the complementary characteristics of these datasets, the HIDIV framework is thoroughly evaluated against both internal and external intrusion threats within IoV environments. For dataset preprocessing, we removed the Timestamp and DLC dimensions from both the Car-Hacking and Survival Analysis datasets, while employing a feature selection algorithm to extract the 20 most significant features from the CICIDS2017 dataset [25]. To ensure robust and unbiased evaluation, a partial random selection of data is performed, followed by a stratified split into a 70% training set and a 30% testing set. The distribution of data across the three datasets is summarized in Table I. Furthermore, to validate the efficacy of the dynamic encoder update mechanism, we evaluated HIDIV against the Baseline HDC—a traditional architecture lacking this dynamic update—on the same datasets, with results confirming HIDIV's superior accuracy.

B. Accuracy Comparison

As illustrated in Fig. 4, HIDIV exhibits excellent classification performance across multiple benchmark datasets. Specifically, on the Car-Hacking dataset, HIDIV-3k achieves an accuracy of 98.67%, closely rivaling DNN and CNN, while outperforming the BaselineHD model operating at the same dimensionality. In the Survival Analysis dataset, HIDIV-5k attains an impressive accuracy of 99.89%, ranking second only to the CNN model. Notably, HIDIV-5k significantly outperforms the BaselineHD model, even when the latter operates at equal or higher dimensional configurations. For

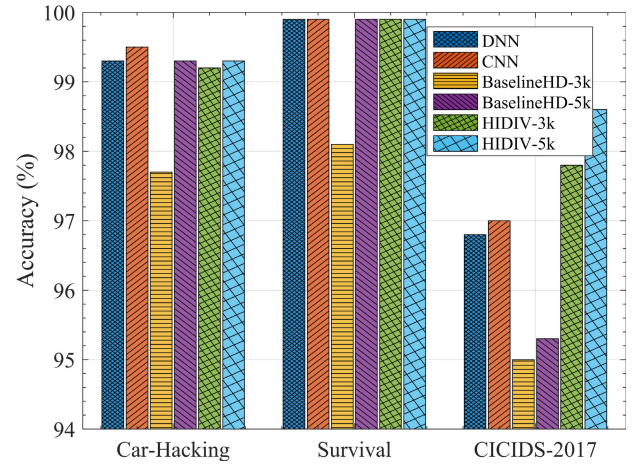


Fig. 4. Accuracy comparison of IDS classification.

the CICIDS2017 dataset, HIDIV-5k achieves the highest performance among all evaluated models, with an accuracy of 98.3%. This result further demonstrates HIDIV's capacity to effectively manage complex, real-world network traffic data and highlights its robustness in handling diverse intrusion detection scenarios.

C. Cluster Accuracy

As shown in Fig. 5, the experimental data demonstrate the differential impact of cluster numbers on model accuracy. For the Car-Hacking dataset, the most significant accuracy degradation of 17.11% occurs when using only 4 clusters. As the cluster count increases to 16, the accuracy recovers to within just 0.03 percentage points of the original performance. The Survival dataset exhibits a 9.62% accuracy drop with 4 clusters, but fully regains its baseline accuracy when expanded to 32 clusters. Notably, the CICIDS2017 dataset shows the greatest sensitivity to cluster compression, suffering a 9.05% accuracy reduction at 4 clusters and requiring 64 clusters to restore its original accuracy level completely. These results indicate that datasets with varying complexity levels require differentiated clustering strategies. Simpler datasets can maintain accuracy with moderate clustering, while more complex datasets demand richer cluster representations to preserve model performance.

D. Execution Efficiency

1) *Convergence Comparison:* As previously discussed, BaselineHD updates only the class hyperdimensional vectors during training, whereas HIDIV further refines the projection matrix in addition to this process. However, as shown in Fig. 6, HIDIV achieves significantly higher classification accuracy than BaselineHD at the same dimensionality. Moreover, the accuracy trends during training indicate that HIDIV converges faster. This suggests that in practical IoV scenarios, HIDIV can attain higher classification accuracy with fewer training epochs, enhancing its efficiency.

2) *Execution Time Comparison:* As illustrated in Fig. 7 and Fig. 8, HIDIV exhibits comparable training and inference

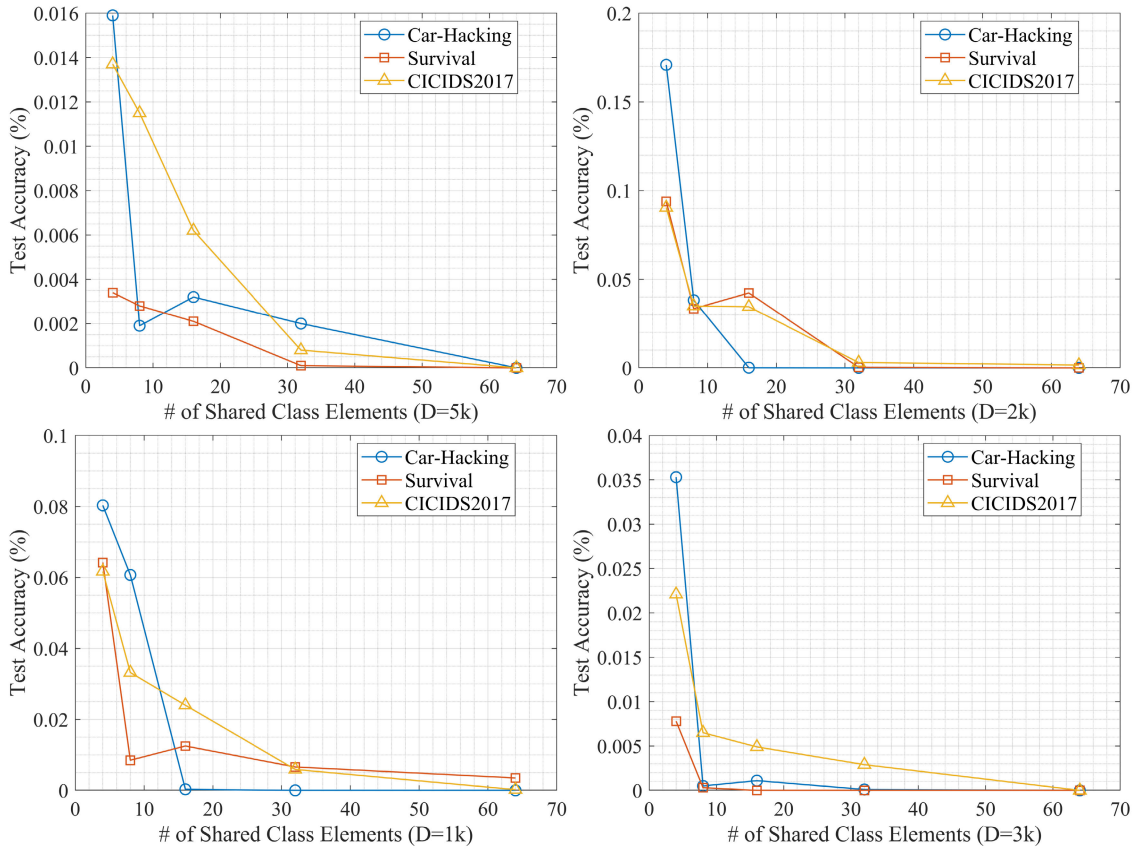


Fig. 5. Impact of number of clusters on the quality loss across different dimensionalities.

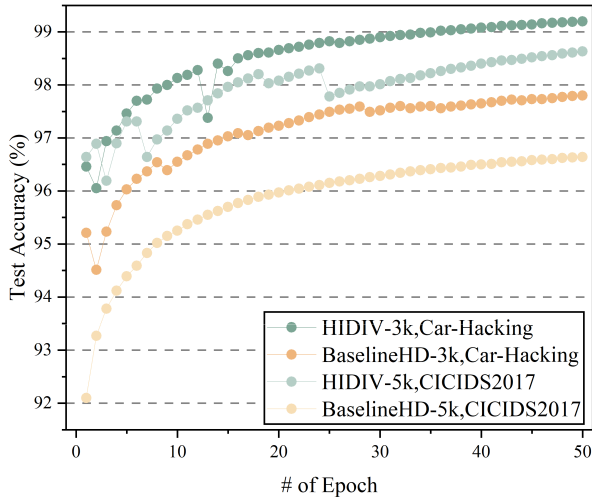


Fig. 6. Training accuracy over epochs for the Car-Hacking and CICIDS2017 datasets.

speeds to BaselineHD at the same dimensionality. For example, on the Car-Hacking dataset, HIDIV-3k's inference time of $7.12 \mu s$ is close to BaselineHD-3k's $7.1 \mu s$, while HIDIV-5k's $7.19 \mu s$ is comparable to BaselineHD-5k's $7.23 \mu s$. Since both methods employ similar inference procedures, their inference times remain comparable. In contrast, HIDIV shows significant speed improvements over DNN and CNN. For instance, on the CICIDS-2017 dataset, HIDIV-5k achieves an inference time of $8.51 \mu s$, approximately 41% faster than DNN's $11.58 \mu s$ and 52% faster than CNN's $11 \mu s$. On average, HIDIV

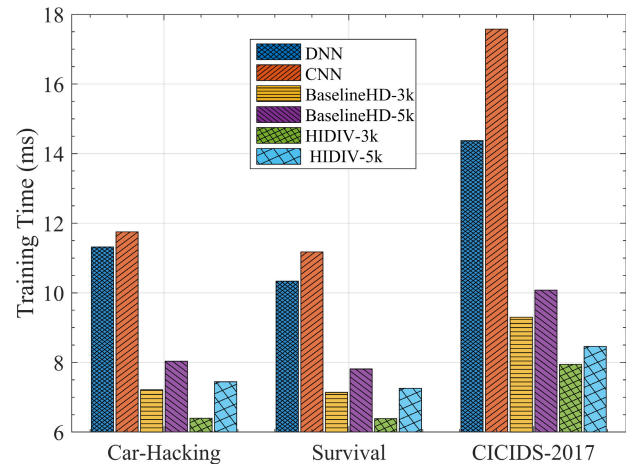


Fig. 7. Comparison of training time on GPU.

achieves 43.4% faster training and inference speeds compared to DNN, and 33.2% faster than CNN. These results highlight HIDIV's ability to deliver both superior accuracy compared to BaselineHD and significantly faster training and inference speeds than traditional deep learning models.

E. Resource Utilization

For the case of $D=5k$, the FPGA resource utilization under different cluster configurations 4, 8, 16, 32 is shown in Tab. II. As discussed previously, in the pipelined architecture, the inference latency is primarily determined by the

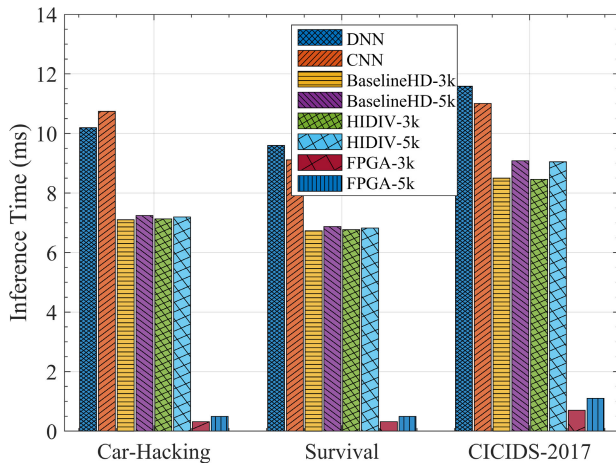


Fig. 8. Comparison of inference time on GPU.

computational delay of the Encoding module. To ensure the generalizability, the proposed acceleration architecture employs a conventional DSP chain for vector multiplication. This design predominantly consumes DSP resources while avoiding complex routing logic and minimizing additional Look Up Table (LUT) overhead. Theoretically, the performance of the Encoding module exhibits a positive correlation with the number of DSPs utilized. However, in resource-constrained edge computing scenarios, the DSP resources of hardware platforms are often strictly limited. To better align with the characteristics of edge devices, this study restricts the DSP allocation for the Encoding module to use 500 DSP units is a configuration achievable on most FPGA development boards. In terms of power consumption, when the number of clusters is 32, deploying the HIDIV accelerator on CICIDS-2017 incurs only 2.356 J of static power consumption, while each inference cycle requires approximately 2.4 mJ. These results demonstrate the suitability of the proposed design for edge deployments with stringent power budgets.

Following clustering analysis, the vector multiplication in similarity computation is replaced by multi-channel addition operations, where the channel parallelism is jointly determined by the number of clusters and classes. Consequently, as the cluster and class quantities increase, the hardware resource overhead of the Associative Search module grows significantly. As detailed in Section, this module primarily utilizes Look-Up Tables (LUTs) and Flip-Flops (FFs) for data bit selection and buffering. Thus, for a fixed dataset, the LUT and Flip Flop (FF) resource consumption demonstrate an approximately linear relationship with the number of clusters. Additionally, the storage requirement for Class Hypervectors is $D \times \log_2 k$ bits, implying that increasing the cluster count directly leads to a linear expansion in BRAM resources. A comparative analysis across datasets demonstrates that under identical cluster configurations, the CICIDS-2017 dataset, with its larger number of classes, demands substantially more LUT, FF, and BRAM resources than the Survival dataset.

F. Robustness Comparison

In real-world IoV environments, noise-induced parameter errors present significant challenges to IDS reliability.

TABLE II
RESOURCE UTILIZATION OF HIDIV INFERENCE FOR SURVIVAL AND CICIDS DATASETS

Dataset	Clusters	LUT	FF	BRAM	DSP	Power(J)
Survival	C-4	4106	19382	10	500	1.078
	C-8	7530	21235	13.5	500	1.201
	C-16	15600	28917	18	500	1.444
	C-32	42382	62199	22.5	500	2.115
CICIDS	C-4	5822	21998	15	500	1.169
	C-8	14041	31755	21	500	1.399
	C-16	31210	50604	27	500	1.833
	C-32	81244	112083	34	500	2.356

Hardware Error			1.0%	2.0%	5.0%	10.0%	15.0%
HIDIV	1-bit	4k	0.10%	0.40%	1.30%	2.40%	3.50%
		3k	1.20%	1.50%	2.40%	3.50%	4.60%
		2k	1.70%	2.00%	2.90%	4.00%	5.10%
		1k	2.90%	3.20%	4.10%	5.20%	6.30%
	2-bit	4k	1.34%	1.05%	2.24%	3.62%	4.25%
		3k	1.07%	1.88%	2.90%	5.90%	6.90%
		2k	1.97%	2.82%	3.32%	6.67%	8.08%
		1k	2.31%	3.21%	3.51%	8.45%	8.84%
	4-bit	4k	1.60%	2.13%	6.17%	11.54%	15.76%
		3k	1.50%	2.89%	6.64%	11.63%	15.47%
		2k	1.61%	3.13%	7.12%	12.36%	16.33%
		1k	3.53%	4.96%	8.79%	13.90%	17.92%
	8bit	4k	1.49%	2.91%	6.63%	11.35%	14.69%
		3k	1.52%	2.95%	6.71%	11.48%	14.82%
		2k	1.92%	3.72%	8.86%	14.35%	15.81%
		1k	1.95%	3.79%	8.56%	14.75%	19.40%

Fig. 9. Comparison of the robustness of HIDIV and DNN-based models under varying error rates.

To evaluate the robustness of HIDIV under such conditions, we conduct a comparative analysis with the SOTA DNN model [33]. For a fair comparison, all DNN model weights are uniformly quantized to 8 bits, aligning with the memory constraints typically found in embedded IoV systems. Robustness is evaluated by introducing random bit flips at varying rates in the model parameters. The error rate is defined as the percentage of bits flipped within the memory representations of both the DNN and HIDIV models.

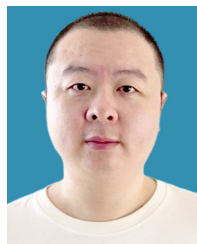
As shown in Fig. 9, HIDIV demonstrates significantly greater resilience to parameter errors compared to the DNN baseline. This enhanced robustness can be attributed to the inherent redundancy and holographic representation of information in high-dimensional hypervectors. For example, under 1-bit quantization, the HIDIV model with 4k dimensions achieves $13\times$ higher robustness than the DNN model. Even under 8-bit quantization, HIDIV maintains its advantage. At a parameter error rate of 15%, HIDIV with 4k-dimensional hypervectors achieves $2.8\times$ higher robustness than the DNN model. In addition, increasing the dimensionality of hypervectors directly enhances robustness. For instance, with a 10% error rate, HIDIV using 4k dimensions at 8-bit precision offers $1.29\times$ greater robustness than its 1k-dimensional variant at the same bit width. These findings confirm that HIDIV not only delivers strong classification performance but also offers high fault tolerance, making it well-suited for deployment in error-prone and resource-constrained IoV environments.

VII. CONCLUSION

This paper presents HIDIV, a novel intrusion detection architecture tailored for the IoV. By incorporating a dynamic encoder adjustment mechanism during training, HIDIV significantly improves training efficiency while achieving superior classification accuracy. The proposed framework effectively addresses both internal and external attack scenarios within IoV systems, and its performance is rigorously evaluated on three representative benchmark datasets. Experimental results demonstrate that HIDIV achieves competitive accuracy comparable to ML-based IDS models. Moreover, HIDIV delivers faster inference times, making it highly suitable for deployment in resource-constrained and real-time IoV environments.

REFERENCES

- [1] D. Tang, H. Lu, Y. Liu, X. He, and S. Zhang, "Hyperdimensional computing for intrusion detection in the Internet of Vehicles," in *Proc. IEEE/CIC Int. Conf. Commun. China (ICCC)*, China, Aug. 2025, pp. 1–6.
- [2] L. Yang, A. Shami, G. Stevens, and S. de Russett, "LCCDE: A decision-based ensemble framework for intrusion detection in the Internet of Vehicles," in *Proc. IEEE Global Commun. Conf.*, Dec. 2022, pp. 3545–3550.
- [3] L. Yang, A. Moubayed, I. Hamieh, and A. Shami, "Tree-based intelligent intrusion detection system in Internet of Vehicles," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.
- [4] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain, "Machine learning-based network vulnerability analysis of industrial Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6822–6834, Aug. 2019.
- [5] J. Azimjonov and T. Kim, "A comprehensive empirical analysis of data sets, regression-based feature selectors, and linear SVM classifiers for intrusion detection systems," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34676–34693, Nov. 2024.
- [6] M. M. Isa and L. Mhamdi, "Hybrid deep autoencoder with random forest in native SDN intrusion detection environment," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 1698–1703.
- [7] W. He, Y. Liu, H. Yao, T. Mai, N. Zhang, and F. R. Yu, "Distributed variational Bayes-based in-network security for the Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6293–6304, Apr. 2021.
- [8] S. Ding, Y. Wang, and L. Kou, "Network intrusion detection based on BiSRU and CNN," in *Proc. IEEE 18th Int. Conf. Mobile Ad Hoc Smart Syst. (MASS)*, Oct. 2021, pp. 145–147.
- [9] R. Jablaoui and N. Liouane, "Efficient RNN models for IoT intrusion detection system," in *Proc. Int. Conf. Control, Autom. Diagnosis (ICCAD)*, May 2024, pp. 1–6.
- [10] L. Yang and A. Shami, "A transfer learning and optimized CNN based intrusion detection system for Internet of Vehicles," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 2774–2779.
- [11] J. Wang, H. Chen, M. Issa, S. Huang, and M. Imani, "Late breaking results: Scalable and efficient hyperdimensional computing for network intrusion detection," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–2.
- [12] R. Wang, F. Kong, H. Sudler, and X. Jiao, "Brief industry paper: HDAD: hyperdimensional computing-based anomaly detection for automotive sensor attacks," in *Proc. IEEE 27th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, May 2021, pp. 461–464.
- [13] G. Karunaratne, A. Rahimi, M. L. Gallo, G. Cherubini, and A. Sebastian, "Real-time language recognition using hyperdimensional computing on phase-change memory array," in *Proc. IEEE 3rd Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, Jun. 2021, pp. 1–16.
- [14] D. Ma, S. Zhang, and X. Jiao, "Robust hyperdimensional computing against cyber attacks and hardware errors: A survey," in *Proc. 28th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2023, pp. 598–605.
- [15] M. Imani, J. Morris, S. Bosch, H. Shu, G. D. Micheli, and T. Rosing, "AdaptHD: Adaptive efficient training for brain-inspired hyperdimensional computing," in *Proc. IEEE Biomed. Circuits Syst. Conf. (BioCAS)*, Oct. 2019, pp. 1–4.
- [16] H. Lee, H. Kwon, J. Kim, S. Kim, M. Imani, and Y. Kim, "Towards forward-only learning for hyperdimensional computing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–2.
- [17] K. Wang, A. Zhang, H. Sun, and B. Wang, "Analysis of recent deep-learning-based intrusion detection methods for in-vehicle network," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 1843–1854, Feb. 2023.
- [18] J. Gao, Y. Lu, Y. He, M. Fan, D. Han, and Y. Qiao, "Tokenization representation and deep-learning-based intrusion detection in Internet of Vehicles," *IEEE Internet Things J.*, vol. 11, no. 23, pp. 37974–37987, Dec. 2024.
- [19] J. Zhang, B. Gong, M. Waqas, S. Tu, and S. Chen, "Many-objective optimization based intrusion detection for in-vehicle network security," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15051–15065, Dec. 2023.
- [20] L. Wang, X. Zhang, D. Li, and H. Liu, "Multi-sensors space and time dimension based intrusion detection system in automated vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 200–215, Jan. 2024.
- [21] J. Qin, Y. Xun, and J. Liu, "CVMIDS: Cloud-vehicle collaborative intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 321–332, Feb. 2024.
- [22] Z. Qu and Z. Cai, "FEDSA-ResnetV2: An efficient intrusion detection system for vehicle road cooperation based on federated learning," *IEEE Internet Things J.*, vol. 11, no. 18, pp. 29852–29863, Sep. 2024.
- [23] Z. Abou El Houda, H. Moudoud, B. Brik, and L. Khoukhi, "Blockchain-enabled federated learning for enhanced collaborative intrusion detection in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7661–7672, Jul. 2024.
- [24] R. Xing, Z. Su, and Y. Wang, "Collaborative intrusion detection approach based on blockchain in Internet of Vehicles," *IEEE Internet Things J.*, vol. 12, no. 9, pp. 11965–11976, May 2025.
- [25] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
- [26] M. A. Elsayed, M. Wrana, Z. Mansour, K. Lounis, S. H. H. Ding, and M. Zulkernine, "AdaptIDS: Adaptive intrusion detection for mission-critical aerospace vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23459–23473, Dec. 2022.
- [27] H. Sun, J. Wang, J. Weng, and W. Tan, "KG-ID: Knowledge graph-based intrusion detection on in-vehicle network," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 4, pp. 4988–5000, Apr. 2025.
- [28] J. Liu, Z. Guan, D. Liu, S. Miao, and F. Dai, "Integrating branching and pruning for efficient hyperdimensional computing," in *Proc. IEEE 42nd Int. Conf. Comput. Design (ICCD)*, Nov. 2024, pp. 699–706.
- [29] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," 2015, *arXiv:1502.01852*.
- [30] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198.
- [31] M. L. Han, B. I. Kwak, and H. K. Kim, "Anomaly intrusion detection method for vehicular networks based on survival analysis," *Veh. Commun.*, vol. 14, pp. 52–63, Oct. 2018.
- [32] Kurniabudi, D. Stiawan, Darmawijoyo, M. Y. Bin Idris, A. M. Bamhdi, and R. Budiarto, "CICIDS-2017 dataset feature analysis with information gain for anomaly detection," *IEEE Access*, vol. 8, pp. 132911–132921, 2020.
- [33] A. Rosay, F. Carlier, and P. Leroux, "MLP4NIDS: An efficient MLP-based network intrusion detection for CICIDS2017 dataset," in *Proc. Int. Conf. Mach. Learn. Netw. (MLN)*, 2020, pp. 240–254.



Xiaoming He (Member, IEEE) received the Ph.D. degree in computer science and software engineering from Hohai University, Nanjing, China, in 2023. He is currently a Lecturer with the College of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing, China. Prior to work, he was a Visiting Research Fellow with Singapore University of Technology and Design. His current research interests include edge intelligence and FPGA-based AI accelerators.



Da Tang received the B.S. degree from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2023, where he is currently pursuing the M.S. degree in information networks. His research interests include hyperdimensional computing acceleration.



Haodong Lu received the Ph.D. degree in signal and information processing from Nanjing University of Posts and Telecommunications, Nanjing, China, in December 2024. He is currently a Post-Doctoral Fellow with Fudan University, Shanghai, China. His research interests include edge intelligence, neural network acceleration, and software/hardware co-design.



Yunzhe Jiang received the B.S. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2024, where he is currently pursuing the M.S. degree with the School of Information and Communication Engineering. His research interests include mobile edge computing.

Hadeel Alsolai received the Ph.D. degree in computer and information sciences from the University of Strathclyde, Glasgow, U.K., in March 2021. She is currently an Assistant Professor with the Department of Information Systems, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University. Also, she has been appointed as the Manager of the Office Business and Project Management, College of Computer and Information Sciences. She teaches several courses with the Information Systems Department, such as information systems, system security, and database systems. Her research interests include machine learning, data mining, data science, software quality, and open-source systems.



Shahid Mumtaz (Senior Member, IEEE) is currently a Professor with Nottingham Trent University (NTU), U.K. He is a scientific expert and an evaluator for various research funding agencies. He has authored four technical books, 12 book chapters, and more than 300 technical papers (more than 200 IEEE journals/IEEE TRANSACTIONS, more than 100 conferences, and two IEEE best paper awards) in mobile communications. Most of his publications are in the field of wireless communication. In 2012, he was awarded the Alain Bensoussan Fellowship.

He received the Young Scientist Fellowship Award in China in 2017. He is an IET Fellow, the Founder, the Editor-in-Chief of *IET Journal of Quantum Communication*, and the Vice-Chair of Europe/Africa Region-IEEE ComSoc: Green Communications and Computing Society.